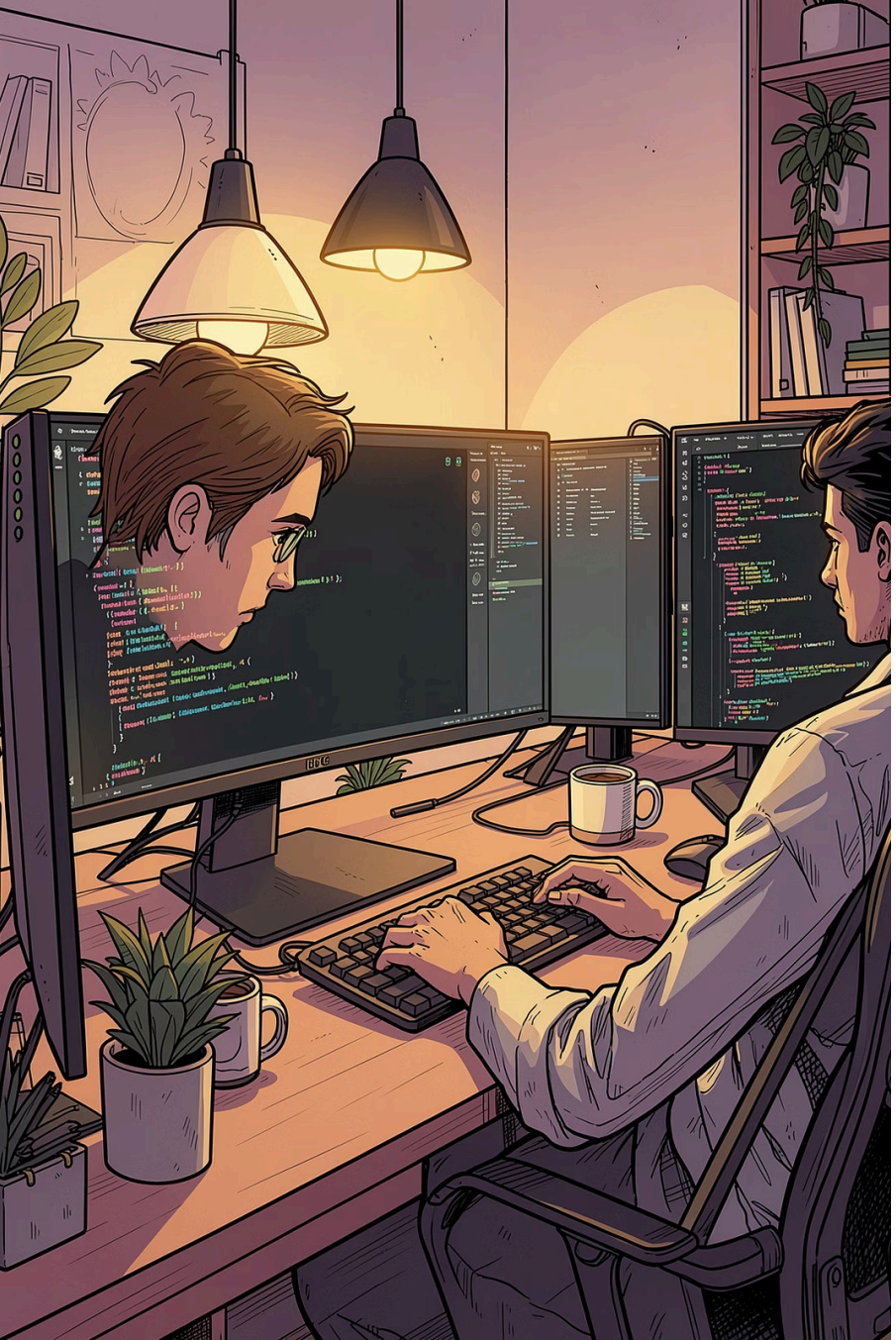


From Thought to Code: Mastering the Programming Logic

Er Piyush Pant

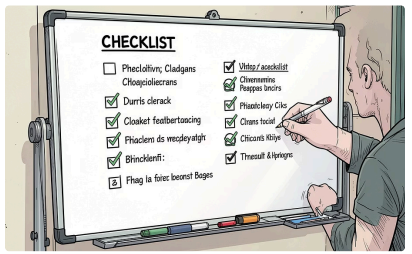
Before a single line of code is written, great software begins with **great thinking**. This presentation explores the essential tools every programmer uses to transform raw ideas into structured, executable solutions using algorithms, flowcharts, and pseudocode.

C++ INTRODUCTION

PROGRAMMING LOGIC

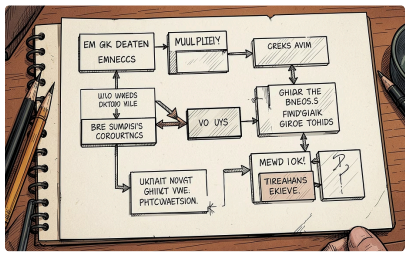
The Programmer's Blueprint

Before writing code, developers plan first. They use three simple tools to organize ideas, show how a program works, and check if their logic makes sense.



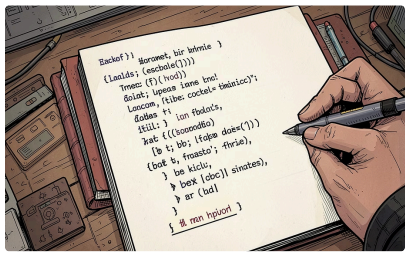
Algorithm

A clear list of steps to solve a problem.



Flowchart

A picture that shows the steps and decisions in a process.



Pseudocode

Simple, code-like notes written in plain language.

What is an Algorithm?

An **algorithm** is a step-by-step procedure to solve a problem.

Think of it like a recipe: it gives clear instructions in the right order so you get the same result every time.

Clear Steps

Each step is easy to follow.

Specific Order

The steps happen one after another.

Solves a Problem

It helps you reach a result.

Example: Cooking pasta is an algorithm: boil water, add pasta, cook for 10 minutes, drain, and serve.

The Three Pillars of Logic

Sequence

Do steps in order.

Example: tie your shoes, then put on your backpack, then leave the house.

Selection

Make a choice with **if/else**.

Example: if it rains, take an umbrella; otherwise, wear sunglasses.

Repetition

Do something multiple times with **loops**.

Example: brush each tooth, one after another, until all are clean.

The Flowchart

A flowchart is a **picture of steps** in an algorithm. It uses simple shapes and arrows to show what happens first, next, and last.

Standard Flowchart Symbols

- **Oval**

  Start or end.

- **Rectangle**

  A step or action.

- **Diamond**

  A question with two answers: yes or no.

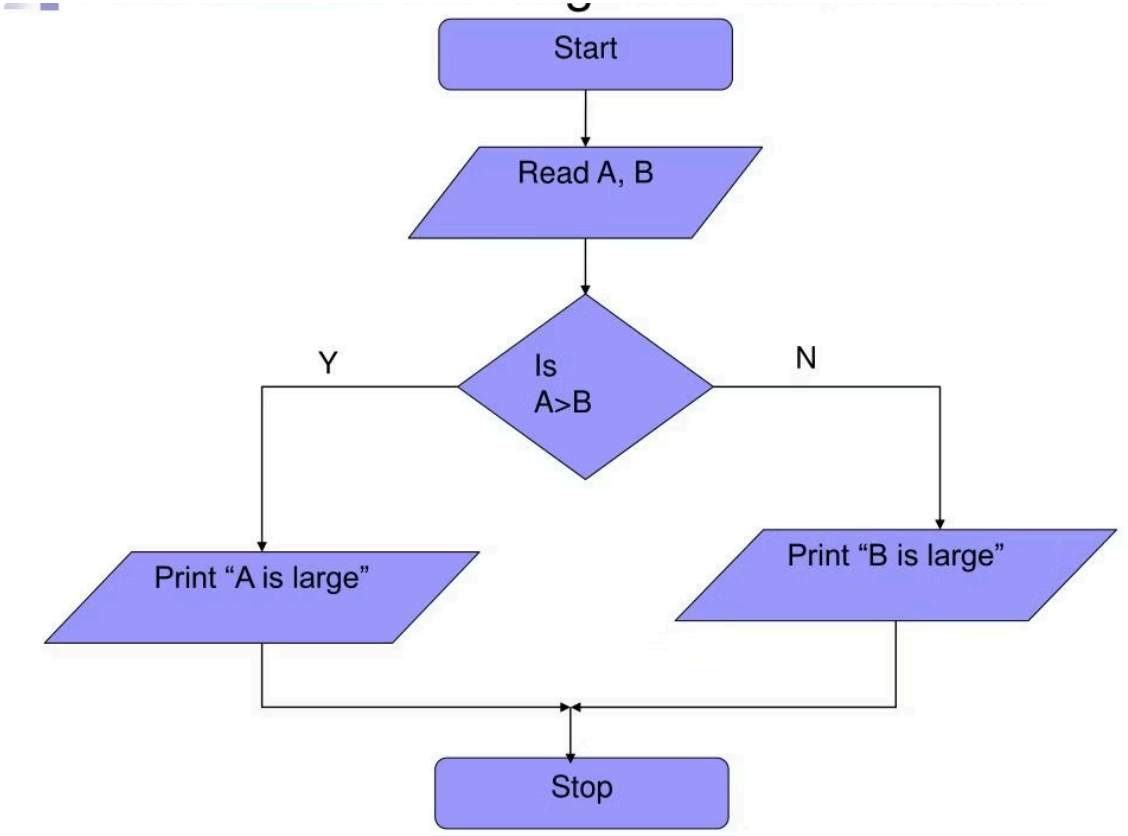
- **Parallelogram**

  Input or output.

- **Arrow**

  Shows the order of the steps.

Example: Find the Larger Number



This example shows how a flowchart can help students follow an algorithm step by step.

Pseudocode

Pseudocode is a **simple way to write an algorithm in plain English**. It helps you plan the steps before you write real code.

Easy to Read

Pseudocode uses simple words, so it is easier to understand than real programming code.

Shows the Steps

Each line tells you what happens next, which makes the logic easy to follow.

Good for Planning

You can use it to think through a problem before choosing a programming language.

Simple pseudocode example: find the larger number

```
READ num1, num2
IF num1 > num2 THEN
  WRITE "num1 is larger"
ELSE
  WRITE "num2 is larger"
ENDIF
```

What each line means:

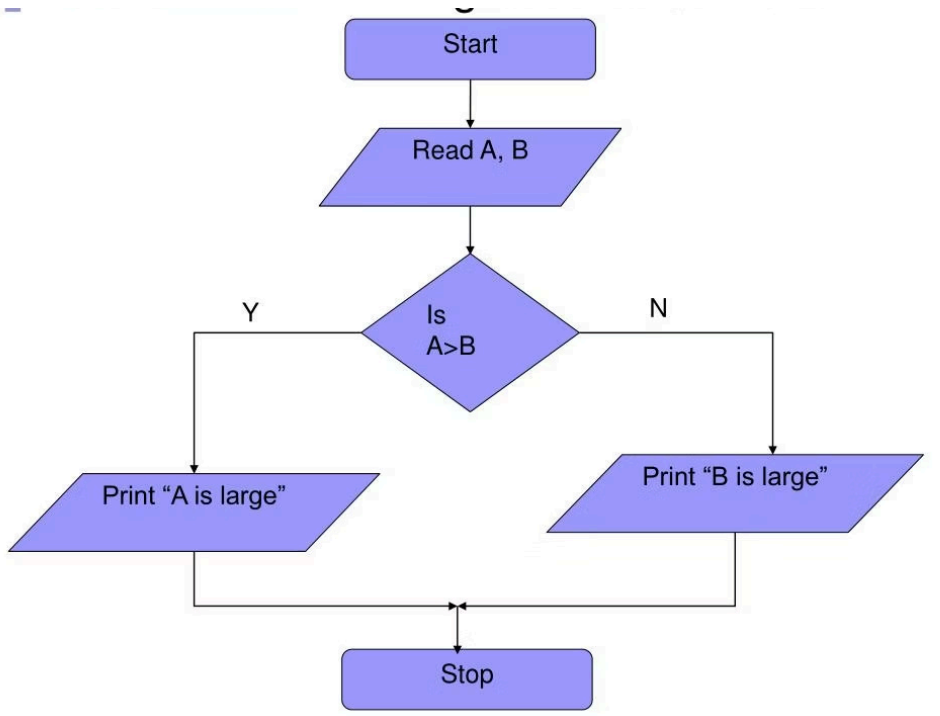
- `READ num1, num2` — get two numbers from the user.
- `IF num1 > num2 THEN` — check whether the first number is bigger.
- `WRITE "num1 is larger"` — show this message if the first number is bigger.
- `ELSE` — otherwise, do the next step.
- `WRITE "num2 is larger"` — show this message if the second number is bigger.
- `ENDIF` — end the decision.

Example 1: Find the Larger Number

Algorithm in Plain English

1. Start the program.
2. Read two numbers into variables A and B.
3. Compare A and B using a conditional check.
4. If A is greater than B, A is the larger number.
5. If B is greater than A, B is the larger number.
6. Display the larger number.
7. Stop the program.

Flowchart



Pseudocode

```
START
  READ A, B
  IF A > B THEN
    WRITE "A is larger"
  ELSE
    WRITE "B is larger"
  ENDIF
STOP
```

C++ Code

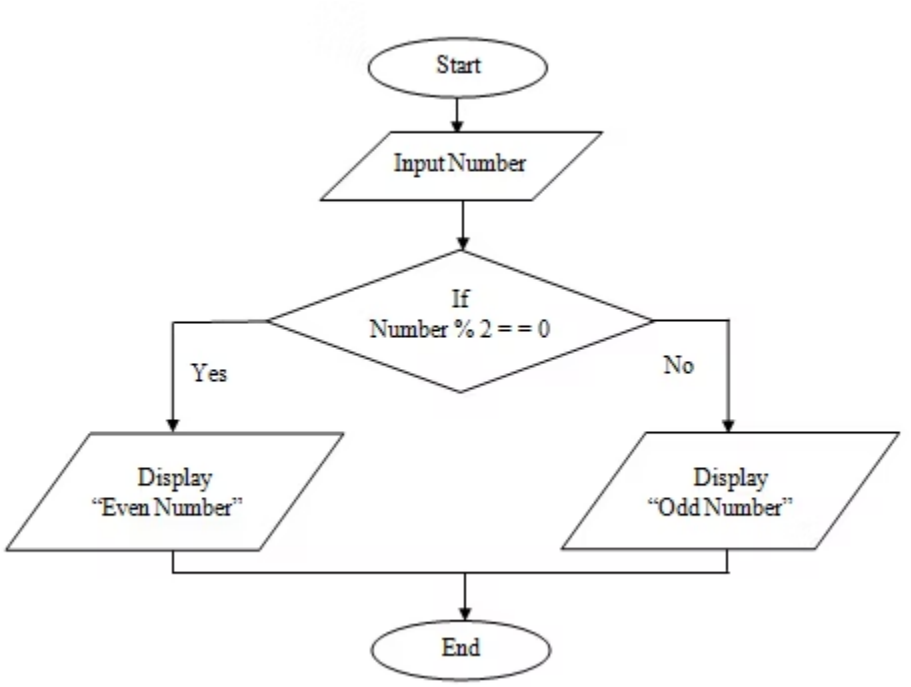
```cpp
#include <iostream>
using namespace std;

int main() {
    // Declare two variables to store the numbers
    int A, B;

    // Ask the user to enter two numbers
    cout << "Enter two numbers: ";
    cin >> A >> B;

    // Compare the two numbers and display the larger one
    if (A > B) {
        cout << "A is larger" << endl;
    } else {
        cout << "B is larger" << endl;
    }

    return 0;
}
```

Example 2: Check if a Number is Odd or Even

Algorithm in Plain English

1. Start the program.
2. Ask the user to enter an integer.
3. Read the number from the keyboard.
4. Check whether the number can be divided by 2 evenly.
5. If the remainder is 0, the number is even.
6. If the remainder is not 0, the number is odd.
7. Display the result to the user.
8. Stop the program.

Flowchart



Pseudocode

```
START
  READ number

  IF number MOD 2 == 0 THEN
    WRITE "Even"
  ELSE
    WRITE "Odd"
  ENDIF

STOP
```

C++ Code

```cpp
#include <iostream>
using namespace std;

int main() {
    // Declare a variable to store the user's input
    int number;

    // Ask the user to enter an integer
    cout << "Enter an integer: ";

    // Read the input value from the keyboard
    cin >> number;

    // Check whether the number is divisible by 2
    if (number % 2 == 0) {
        // If the remainder is 0, the number is even
        cout << number << " is Even." << endl;
    } else {
        // Otherwise, the number is odd
        cout << number << " is Odd." << endl;
    }

    return 0;
}
```

Putting It Into Action: The Implementation Cycle

Software development is easier when you follow a simple cycle. For students, it helps turn an idea into working code one step at a time.

01

Plan

Start with a flowchart and pseudocode so you know what the program should do before you write code.

02

Write Code

Turn the plan into C++ code. This is where your idea becomes a program.

03

Test It

Run the program with different inputs to see if it works the way you expected.

04

Fix Errors

If something goes wrong, debug and improve it until the program works correctly.

Each step matters because it saves time, reduces mistakes, and makes your code more reliable.

Conclusion: Plan Well, Code Better

> Good planning makes coding easier. When you master tools like flowcharts and pseudocode, you gain the confidence to solve any programming problem.

Think First

Take time to understand the problem before you start coding.

Visualize with Flowcharts

Map out your logic so the path to a solution is clear.

Draft with Pseudocode

Outline your solution in simple steps before writing code.

Code with Confidence

With a strong plan, turning ideas into code becomes much easier.